SIGEC CLOUD · CLÍNICA SAN PEDRO

DOCUMENTO DE ARQUITECTURA Y OPERACIÓN

Documentación técnica

Especificación del sistema de gestión clínica: arquitectura, módulos, modelo de datos, seguridad, despliegue y plan de evolución.

EDICIÓN	FECHA DE CORTE	CLASIFICACIÓN
1.0 · Basada en el repositorio inspeccionado	11 de octubre de 2026	Documentación técnica interna

01 · VISIÓN GENERAL

Resumen ejecutivo y alcance

SIGEC Cloud es una aplicación web de gestión de consultorio médico. Centraliza expediente clínico, agenda, caja/pagos e información administrativa mediante una SPA en React conectada directamente a Supabase.

Aplicación web

React 18, Vite 5, React Router 6 y CSS propio; renderizado de cliente.

Servicios gestionados

Supabase Auth, PostgreSQL, Row Level Security, Realtime y Edge Functions.

Perfiles operativos

Roles almacenados en el perfil: médico y secretaria.

Despliegue web

Netlify sirve los artefactos estáticos de Vite con fallback SPA.

Alcance implementado

- Inicio/cierre de sesión, carga de perfil, protección básica de rutas y estado activo de cuenta.
- Dashboard con resumen diario de citas, sala de espera y recaudación; accesos a pacientes, agenda, pagos y configuración.
- Pacientes y expediente individual con información general, secciones clínicas, evoluciones, informes médicos imprimibles y plantillas de plan/tratamiento.
- Agenda/citas con estados, paciente registrado o texto libre, métodos de pago y registro de cobros.
- Pagos con consulta de registros e informe contable por período para el perfil médico.
- Configuración de tasa BCV y administración de usuarios/accesos para el médico.

> **Límite importante:** este documento describe lo encontrado en el código fuente y migraciones del repositorio. No certifica cumplimiento normativo, calidad clínica, disponibilidad, configuración real del proyecto Supabase ni políticas del proveedor. Antes de usarlo con datos reales debe completarse una revisión de seguridad y privacidad.

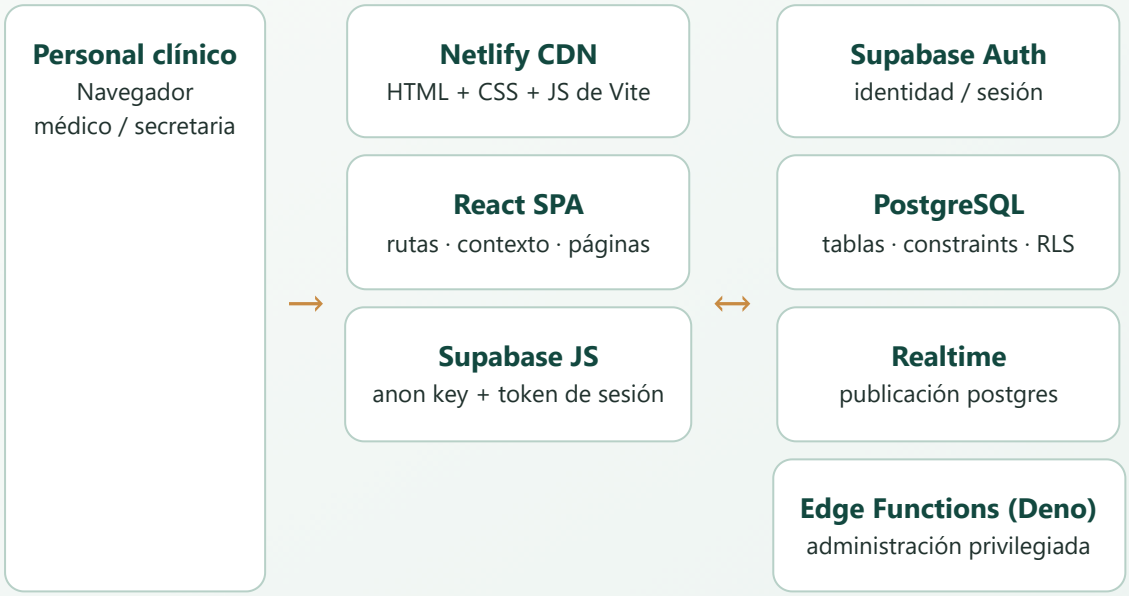
Convenciones de precisión

Implementado comportamiento visible en código o SQL. **Observación** diferencia o limitación comprobada. **Recomendación** mejora propuesta no asumida como existente.

02 · DISEÑO

Arquitectura lógica

La interfaz es una SPA: React se monta en el navegador, el enrutador selecciona una vista y los componentes consultan Supabase usando el cliente JavaScript. No se observa una API HTTP propia intermedia para las operaciones ordinarias de pacientes, citas o pagos.



El cliente usa credenciales públicas del proyecto y la sesión del usuario. La clave service_role debe permanecer exclusivamente en el entorno seguro de funciones.

Responsabilidades

CAPA	RESPONSABILIDAD	IMPLEMENTACIÓN OBSERVADA
Presentación	Formularios, tablas, navegación, modales, impresión.	Componentes React y CSS en `src/`.
Estado de sesión	Usuario, perfil, rol, carga, login/logout.	`AuthContext` suscrito a cambios de Auth.
Acceso a datos	CRUD y consultas directas PostgREST / Auth.	Instancia única Supabase JS importada por páginas.
Persistencia y autorización	Datos clínicos, relaciones, constraints y filtros de fila.	PostgreSQL y políticas RLS en migraciones.
Operaciones administrativas	Crear identidades y activar/desactivar acceso.	Funciones Edge con clave administrativa.
Publicación web	Compilación y fallback de rutas.	Vite build → `dist/` → Netlify.

Estilo de interfaz

La hoja global define una paleta verde institucional, acentos dorados, tarjetas/tablas/modales y estilos de impresión en tamaño Carta para informes clínicos. El diseño usa CSS propio, iconos `lucide-react`, layouts flexibles y algunos breakpoints responsive.

03 · CÓDIGO FUENTE

Estructura del repositorio

```
sigec-26/
├── index.html · punto de entrada HTML
├── package.json · dependencias y scripts Vite
├── vite.config.js · plugin React, puerto 3000
├── netlify.toml · build dist y fallback SPA
├── .env.example · variables públicas Supabase (placeholders)
├── src/
│ ├── main.jsx · montaje React StrictMode
│ ├── App.jsx · BrowserRouter y rutas protegidas
│ ├── index.css · diseño global, responsive e impresión
│ ├── components/ · LogoMark, Modal, sidebar, topbar
│ ├── context/AuthContext.jsx · sesión, perfil y roles
│ ├── lib/ · cliente Supabase y formato de fechas
│ └── pages/ · login, Dashboard, pacientes, detalle, Agenda, Pagos, Configuracion
└── supabase/
├── functions/ · create-user, manage-user-access, CORS compartido
└── migrations/ · 001 a 008, esquema y evolución incremental
```

Dependencias declaradas

PAQUETE	USO	RANGO DECLARADO
react / react-dom	Interfaz y renderizado.	^18.2.0
react-router-dom	Rutas del cliente.	^6.22.3
@supabase/supabase-js	Auth, base de datos, funciones y realtime.	^2.39.8
lucide-react	Iconografía.	^0.344.0
vite / @vitejs/plugin-react	Dev server y compilación.	^5.1.6 / ^4.2.1

Las versiones efectivas pueden diferir dentro de los rangos; el archivo lock es la fuente de resolución reproducible de npm.

04 · NAVEGACIÓN Y SESIÓN

Rutas y autenticación

RUTA	VISTA	PROTECCIÓN DE FRONTEND
`/login`	Inicio de sesión.	PublicRoute redirige a / si ya hay usuario.
`/`	Dashboard.	PrivateRoute exige usuario autenticado.
`/pacientes`	Listado/alta de pacientes.	Privada por sesión.
`/pacientes/:id`	Expediente del paciente.	Privada por sesión.
`/agenda`	Citas y caja.	Privada por sesión.
`/pagos`	Pagos e informe contable.	Privada por sesión.
`/configuracion`	Tasa BCV y usuarios.	Privada; el enlace del menú se muestra al médico.

Ciclo de autenticación

1. `AuthProvider` consulta la sesión persistida con `supabase.auth.getSession()`.
2. Si existe identidad, consulta el registro `profiles` por su UUID y determina `rol`.
3. Suscribe `onAuthStateChange` para sincronizar el usuario tras login/logout/refresco.
4. Las rutas muestran carga inicial; después permiten el árbol privado si `user` existe.
5. Cuando lee `is_active=false`, limpia el perfil/usuario y difiere `signOut()`.

> **Observación:** el guard de ruta comprueba solo la identidad, no que exista un perfil válido o un rol reconocido. El acceso efectivo a la base depende de RLS; la interfaz no es por sí sola una frontera de seguridad.

05 · FUNCIONALIDAD

Módulos del sistema

Dashboard

Consulta citas del día ordenadas por hora y pagos del día. Calcula total de citas, pacientes con estado “En sala de espera” y suma de pagos en USD. La acción “Atender” cambia el estado a “Completada” y registra la marca temporal. La cifra recaudada se muestra al médico. El día de consulta se calcula en el navegador con fecha ISO UTC.

Pacientes y expedientes

Permite alta de paciente, listado alfabético, búsqueda en memoria por nombre/cédula/historia y navegación a detalle. Historia es asignada por trigger al insertar. El detalle organiza información general, examen funcional, examen físico, evoluciones, informes y diagnóstico/tratamiento. El expediente clínico se guarda en columnas JSONB para secciones variables. Ofrece impresión de receta/informe; el texto insertado se escapa antes de construir el HTML imprimible.

Agenda

Registra cita para un paciente seleccionado o un nombre escrito libremente. Conserva algunos datos de contacto duplicados en la cita. Permite revisar todas las citas, cambiar estatus, iniciar pago y alternar a la tabla de pagos. Estados: Pendiente, En sala de espera, Confirmada, Completada, Cancelada, No asistió. Método de pago de cita admite Pendiente de Pago, Pago Móvil, Zelle, Efectivo USD, Efectivo Bs, Transferencia y Exonerado.

Pagos y caja

Permite asociar un cobro opcionalmente a paciente, método, monto, fecha, descripción y referencia. Los métodos son Pago Móvil, Zelle, Efectivo USD, Efectivo Bs, Transferencia y Exonerado. Para Exonerado se guarda monto cero. La vista de consulta lista movimientos; el médico puede filtrar por fechas, ver totales/agrupación por método e imprimir un informe contable.

Configuración

Consulta la fila única de configuración con ID 1. La interfaz modifica la tasa BCV; otros metadatos del consultorio están en el esquema, pero no se actualizan en el formulario observado. El panel médico permite crear usuarios con rol, y habilitar/inhabilitar cuentas conservando el historial.

Plantillas e informes médicos

Desde el expediente, el médico puede guardar plantillas reutilizables de plan/tratamiento y crear informes médicos con fecha y texto libre. La impresión incorpora identidad textual de la clínica y datos del paciente. No se encontró almacenamiento de archivos adjuntos ni firma electrónica.

06 · PERSISTENCIA

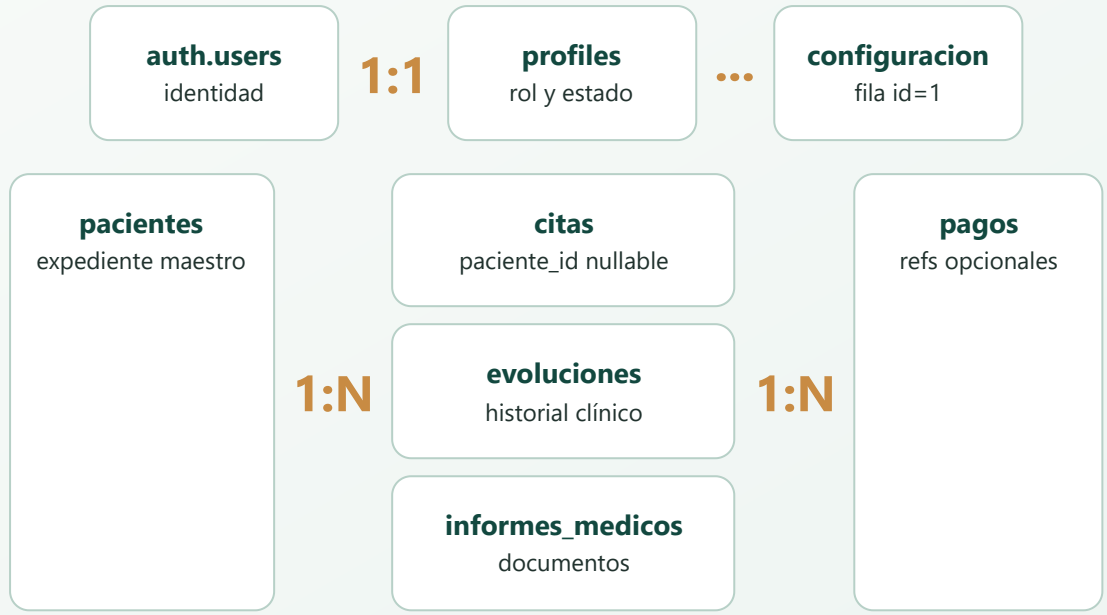
Modelo de datos

La fuente de verdad del esquema es el conjunto secuencial de migraciones `supabase/migrations/001…008`. UUID se habilita mediante la extensión `uuid-ossp`. Se describen las columnas principales; los tipos listados reflejan el SQL.

Entidades y campos

TABLA	CAMPOS RELEVANTES	CONSTRAINTS Y SEMÁNTICA
profiles	`id uuid`, `nombre text`, `rol text`, `created_at`, `is_active boolean`, `disabled_at`, `disabled_by uuid`.	`id` referencia `auth.users` con borrado en cascada. Roles permitidos médico/secretaria; activo predeterminado true.
configuracion	`id int`, `tasa_bcv numeric(10,2)`, `medico_nombre`, `especialidad`, `mpps`, `colegio_medicos`, `rif`, `direccion_clinica`, `updated_at`.	Check `id=1`; una fila semilla. Datos de configuración global.
pacientes	`id uuid`, `historia`, `cedula`, `nombres`, fechas, teléfonos, correo, ocupación, dirección, tres JSONB: `examen_funcional`, `examen_fisico`, `planteamiento`, marcas temporales.	`historia` unique/not null; nombres obligatorio. Trigger genera HC-#### si historia llega vacía. JSONB default {}.
evoluciones	`id`, `paciente_id`, fecha/hora, motivo, `signos_vitales jsonb`, síntomas, examen breve, diagnóstico, plan, observaciones, próxima cita, `created_by`, timestamps.	Paciente obligatorio, cascade delete; motivo obligatorio; autor opcional referenciado a perfil. Signos vitales admiten PA, FC, FR, temperatura, peso, talla e IMC.
citas	`id`, `paciente_id`, nombre y datos copiados del paciente, fecha/hora, motivo, estatus, método, `monto_usd`, `tasa_bcv`, exonerado, atendida_en/por, created_at.	Paciente nullable con SET NULL. Fecha y hora obligatorias. Check de estados y métodos enumerados; importe USD default 0.
pagos	`id`, referencias opcionales a cita/paciente, nombre paciente, método, monto, fecha, descripción, referencia, registrado_por, created_at/updated_at.	Método obligatorio con check; monto >= 0 y default 0; descripción/nombre obligatorios. Fecha por defecto actual. Índices por fecha y cita.
plantillas	`id`, nombre, plan, tratamiento, activo, created_by, created_at/updated_at.	Nombre obligatorio; trigger actualiza `updated_at`. Médicos escriben/borran; autenticados leen según RLS.
informes_medicos	`id`, paciente_id/nombre/cédula, fecha_emision, informe, created_by, timestamps.	Paciente obligatorio con cascade; nombre/informe obligatorios. Índices paciente y fecha. Solo médico puede crear/actualizar según políticas.

Relaciones principales



Una cita puede tener pagos relacionados; no existe constraint que exija que pago/cita/paciente correspondan entre sí. Pagos también pueden no relacionarse con una cita.

Migraciones versionadas

VERSIÓN	CAMBIO
001_init_schema	Extensión, perfiles, configuración, pacientes, secuencia/trigger de historia, evoluciones, citas, RLS inicial.
002_add_cita_patient_details	Agrega datos de contacto/ocupación copiados en citas.
003_enable_realtime	Agrega citas, pacientes, evoluciones y configuración a publicación Realtime si no están agregadas.
004_add_plantillas	Plantillas, trigger de actualización y políticas médico/lectura.
005_add_pagos	Pagos, índices y políticas iniciales.
006_add_informes_medicos	Informes médicos, índices y políticas iniciales.
007_user_access_status	Estado activo, fecha y autor de inhabilitación; índice.
008_enforce_active_accounts	Función is_active_user y revisión de RLS para comprobar cuenta activa.

07 · SEGURIDAD

Autorización, RLS y límites actuales

El cliente usa la URL de Supabase y la clave *anon* expuestas como variables Vite, lo cual es normal solo si RLS está correctamente configurado. El token de sesión identifica al usuario. Las Edge Functions usan `SUPABASE_SERVICE_ROLE_KEY`, que nunca debe estar en el bundle web.

Matriz observada

RECURSO	LECTURA	ESCRITURA / BORRADO SEGÚN RLS
profiles	Autenticado: su propio perfil o todos si cuenta activa.	No se define política de escritura de cliente en estas migraciones; altas/accesos por función admin.
configuracion	Solo usuario autenticado y activo.	UPDATE únicamente médico activo; no se ve política INSERT/DELETE.
pacientes	Todo usuario autenticado activo tiene política FOR ALL (CRUD completo); el alcance no restringe por rol.	
citas	Todo usuario autenticado activo tiene política FOR ALL (CRUD completo); el alcance no restringe por rol.	
evoluciones	Todo usuario autenticado activo tiene política FOR ALL (CRUD completo); el alcance no restringe por rol.	
pagos	Usuarios activos autenticados pueden leer.	INSERT y UPDATE permitidos; no hay política DELETE en las migraciones revisadas.
plantillas	Usuarios activos autenticados.	Crear/actualizar/borrar solo médico activo.
informes_medicos	Usuarios activos autenticados.	Crear/actualizar solo médico activo; no se define DELETE.

> **Hallazgo de riesgo alto:** que una acción esté oculta en la interfaz no la vuelve segura. Por ejemplo, el borrado de pacientes está condicionado por rol médico en el componente, pero la política RLS de pacientes permite FOR ALL a cualquier usuario autenticado activo; de manera similar, la actualización general del expediente y los cambios de cita no quedan limitados a médico por RLS. Revisar la autorización a nivel de base antes de producción.

Controles implementados

- Las rutas del cliente requieren una sesión Supabase.
- La aplicación cancela una sesión cuando detecta `profiles.is_active=false`.
- La función SQL `is_active_user()` es SECURITY DEFINER con `search_path=public` y RLS desactivado dentro de la función; retorna falso si no encuentra perfil activo.
- Las políticas actualizadas para recursos protegen operaciones con identidad autenticada y perfil activo.
- El médico activo es verificado dentro de Edge Functions antes de administrar cuentas y dentro de RLS para config/plantillas/informes.
- Al inhabilitar, se actualiza perfil y se banea la identidad Auth; se intenta restaurar el perfil si falla el ban. No se permite inhabilitarse uno mismo ni inhabilitar al último médico activo.

Controles pendientes recomendados

1. Definir permisos CRUD explícitos por rol y por tabla; separar lectura, alta, actualización y borrado.
2. Evaluar separar datos identificativos de datos clínicos y restringir lectura de evoluciones/informes al médico.
3. Asegurar que cada fila nueva/actualizada mantiene coherencia entre paciente, cita y perfil de autor.
4. Validar perfil activo/rol en guard de aplicación, pero mantener RLS como autoridad final.
5. Revisar CORS de funciones (actualmente origen comodín) y limitarlo a dominios de producción si el flujo lo permite.
6. Definir política de contraseñas, recuperación, rotación, MFA para cuentas médicas y proceso de baja/alta documentado.
7. Establecer clasificación de datos, retención, copias cifradas, auditoría de accesos y respuesta ante incidentes.

> **Claves:** `VITE_SUPABASE_ANON_KEY` es una clave pública pensada para cliente y no reemplaza RLS. `SUPABASE_SERVICE_ROLE_KEY` omite controles normales de RLS y debe guardarse solo en secrets de Supabase Functions/servidor.

08 · ACTUALIZACIÓN DE DATOS

Supabase Realtime

Las páginas crean canales Postgres Changes y eliminan el canal al desmontarse. Ante eventos vuelven a consultar filas usando funciones de fetch. La migración 003 incorpora explícitamente las siguientes tablas a `supabase_realtime`:

citas | pacientes | evoluciones | configuracion

Suscripciones en la interfaz

VISTA	EVENTOS ESCUCHADOS EN FRONTEND	ESTADO DE PUBLICACIÓN OBSERVADO
Dashboard	citas, pagos	citas sí; pagos no añadida en migration 003.
Agenda	citas, pagos, pacientes	citas/pacientes sí; pagos no añadida.
Pacientes	pacientes	Sí.
Detalle paciente	pacientes, evoluciones, informes_medicos, plantillas	pacientes/evoluciones sí; informes/plantillas no añadidas.
Configuración	configuracion	Sí.
Pagos	No se observa suscripción en esa página.	Consulta inicial/manual tras operación.

> **Posible discrepancia:** el cliente escucha cambios en pagos, informes y plantillas, pero la migración de publicación no los agrega. La señalización en tiempo real para esas tablas depende de la configuración desplegada aparte; debe verificarse con Supabase y corregirse explícitamente. La vista Pagos recarga después de acciones propias, no por cambios remotos.

La entrega Realtime no sustituye una consulta inicial: las vistas primero cargan datos y luego se suscriben. El dashboard actualiza su agregación al recibir eventos de citas/pagos solo si la publicación los entrega.

09 · BACKEND PRIVILEGIADO

Supabase Edge Functions

Las funciones están escritas en TypeScript para Deno y comparten cabeceras CORS desde `_shared/cors.ts`. Ambas atienden preflight OPTIONS y devuelven JSON.

`create-user`

1. Requiere encabezado Authorization.
2. Usa URL y service role desde secretos de entorno; verifica la sesión con `auth.getUser`.
3. Busca rol actual en `profiles` y exige médico.
4. Valida email, contraseña, nombre y rol (contraseña mínima de seis caracteres).
5. Crea usuario Auth con email ya confirmado y luego inserta perfil.
6. Si falla el perfil intenta borrar la identidad creada; devuelve identificador/nombre/rol al éxito.

`manage-user-access`

Exige médico activo, valida usuario objetivo y booleano `active`; impide auto-inhabilitación y bloquear al último médico activo. Marca perfil/fecha/actor y aplica baneo Auth de larga duración al inhabilitar. Si el baneo falla intenta revertir el perfil. La habilitación limpia metadatos de baja y elimina el ban.

> **Recomendación:** añadir límites de tasa/abuso, validación de esquemas de request, registro de auditoría sin secretos ni datos clínicos, pruebas de autorización de cada ruta y restricciones CORS por dominio. Definir explícitamente el comportamiento esperado de errores parciales.

10 · INTEGRACIÓN

Configuración y dependencias externas

Variables de entorno del frontend

NOMBRE	USO	CLASIFICACIÓN
`VITE_SUPABASE_URL`	URL del proyecto Supabase para createClient.	Identificador del servicio.
`VITE_SUPABASE_ANON_KEY`	Clave pública anon del proyecto para operaciones cliente sometidas a RLS.	Publicable; restringida por RLS, nunca sustituye políticas.
`SUPABASE_URL`	URL del proyecto para Edge Functions.	Secreto/configuración de función.
`SUPABASE_SERVICE_ROLE_KEY`	Operaciones administrativas Auth/DB dentro de Edge Functions.	Secreto crítico; no exponer en frontend, logs ni documentación.

El archivo `.env.example` contiene placeholders para las dos variables VITE. No se incluyen valores del archivo local de entorno en este documento.

Impresión

La aplicación compone documentos HTML para receta, informe médico e informe contable y llama `window.print()`. Los estilos `@media print` definen páginas Carta. El usuario puede elegir impresora o "Guardar como PDF" en el diálogo del navegador. No es un servicio de PDF firmado/archivado.

Fuera del alcance visto

- No hay dependencia para envío de correo, SMS, WhatsApp ni recordatorios automáticos.
- No se encontró almacenamiento de documentos en Supabase Storage.
- No se observa integración de pago en línea, firma digital o interoperabilidad médica externa.
- No hay servidor de aplicación propio; las operaciones cliente se canalizan mediante Supabase y las dos funciones administrativas.

11 · COMPILACIÓN Y PUBLICACIÓN

Desarrollo, build y despliegue

Requisitos

- Node.js/npm compatibles con las versiones del lock y Vite 5.
- Proyecto Supabase configurado con Auth, PostgreSQL y publicación Realtime.
- Variables frontend apuntando al proyecto correcto; secretos administrativos configurados en Supabase Functions.
- Aplicar las migraciones en orden en el proyecto destino.

Scripts npm declarados

SCRIPT	ACCIÓN	RESULTADO ESPERADO
`npm install`	Instala dependencias desde package/lock.	node_modules
`npm run dev`	Servidor de desarrollo Vite.	Puerto 3000.
`npm run build`	Compila frontend optimizado.	Directorio dist/.
`npm run preview`	Sirve el build local para inspección.	Vista previa estática Vite.

Pruebas automatizadas: en `package.json` no se declara script de test ni suite automatizada. Debe incorporarse validación unitaria/integración y pruebas de políticas RLS antes de cambios de producción.

Netlify

`netlify.toml` define `npm run build`, publica `dist` y redirige cualquier ruta a `/index.html` con estado 200 para soportar navegación SPA. Configurar las dos variables VITE en el entorno de build de Netlify y desplegar funciones/migraciones por el flujo de Supabase del equipo.

Secuencia de publicación recomendada

1. Revisar cambios de migración y permisos con respaldo previo.
2. Aplicar migraciones primero en entorno de pruebas; comprobar esquema, políticas y datos iniciales.
3. Desplegar funciones y asignar secretos de forma segura.
4. Configurar variables del frontend por ambiente y compilar.
5. Ejecutar smoke tests con médico y secretaria, incluyendo intentos de acceso denegado.
6. Promover a producción con monitoreo de errores, Auth, función y consultas.

12 · OPERACIÓN DE NEGOCIO

Flujos funcionales principales

Alta y atención del paciente

1. Usuario con sesión registra nombre y datos demográficos básicos.
2. PostgreSQL asigna UUID e historia clínica correlativa desde una secuencia.
3. El expediente se abre por identificador; el médico completa secciones clínicas.
4. Una evolución referencia al paciente y al perfil que la crea; signos vitales se guardan en JSONB.
5. El médico puede crear informe, usar plantilla y producir impresión local.

Ciclo de cita y cobro

1. La secretaria o médico registra una cita para paciente existente o captura nombre libre.
2. Se almacena fecha, hora, motivo, estatus inicial y datos de cobro previstos.
3. El estado cambia desde Agenda o el dashboard; “Atender” en el dashboard marca completada.
4. Se registra un pago como movimiento separado, con referencia opcional a cita y paciente.
5. El médico genera informe de ingresos de un intervalo y puede imprimirlo.

> **Distinción contable:** `citas.monto_usd` es el monto planificado/relacionado con la cita; `pagos.monto` es el ingreso registrado. El dashboard suma pagos, no los montos de citas.

Ciclo de acceso de usuarios

1. Médico activo crea cuenta y perfil desde Configuración mediante función protegida.
2. El colaborador inicia sesión con credenciales asignadas.
3. El médico puede inhabilitar la cuenta desde la lista; se conserva la fila de perfil e historial de negocio, pero se banea Auth.
4. Al reactivar, se elimina el ban y el perfil se marca activo.

El código crea la identidad con email confirmado y contraseña temporal suministrada. El flujo de cambio obligatorio de contraseña inicial no se observa implementado.

13 · REVISIÓN TÉCNICA

Hallazgos, riesgos y plan recomendado

PRIORIDAD	HALLAZGO DEL REPOSITORIO	ACCIÓN RECOMENDADA
P0 · Antes de datos reales	RLS concede CRUD amplio sobre pacientes, citas y evoluciones a cualquier usuario activo autenticado; algunas acciones se esconden solo en UI.	Definir matriz de permisos por rol y acción, escribir políticas restrictivas por operación; probar directamente con JWT de cada rol.
P0 · Antes de datos reales	Se almacenan datos identificativos y clínicos sensibles.	Revisión jurídica/privacidad aplicable, clasificación, acceso mínimo, respaldo/restauración probados, cifrado y proceso de incidentes.
P1 · Realtime	Frontend escucha pagos, plantillas e informes; migration 003 no publica esas tablas. Pagos tampoco tiene suscripción en su propia página.	Alinear suscripciones/publicación, agregar tablas requeridas y prueba end-to-end multiusuario.
P1 · Autenticación	Guard de navegación confirma sesión, no perfil válido/rol. Error al obtener perfil puede dejar usuario autenticado con perfil nulo en interfaz.	Estados separados (auth/perfil), cerrar o limitar acceso ante perfil ausente y pruebas de estado activo tras cambios.
P1 · Integridad	No existe unicidad de cédula ni reglas de coherencia de pagos con cita/paciente.	Definir reglas de negocio, constraints e índices tras revisar duplicados y datos existentes.
P1 · Auditoría	Autor/fechas existen en algunas tablas, pero no hay bitácora inmutable completa de cambios clínicos/administrativos.	Auditoría central de actor, operación, fecha y entidad sin registrar secretos; retención y acceso acotados.
P2 · Operación	No se declaran pruebas ni pipeline CI; faltan tests de RLS, Edge Functions y flujos.	Implementar lint/build/tests, pruebas automatizadas contra Supabase local/staging y smoke test de despliegue.
P2 · Consistencia	No todas las tablas mantienen automáticamente updated_at; configuración guarda tasa BCV desde la vista actual.	Triggers compartidos, zona horaria de negocio definida, controles para importes y conversión declarada.
P2 · Rendimiento	Listas leen datos completos y búsqueda de pacientes se filtra en memoria; pocos índices adicionales observados.	Paginación, filtros servidor, índices medidos con EXPLAIN y límites para tablas crecientes.

Roadmap propuesto

1. **Seguridad y privacidad:** RLS mínimo privilegio, MFA médico, ciclo de cuenta, auditoría y revisión de tratamiento de datos.
2. **Fiabilidad:** pruebas de políticas/funciones, validación de esquema, manejo de errores uniforme, CI y ambiente staging.
3. **Integridad clínica/financiera:** constraints/relaciones consistentes, timestamps, zona horaria y reglas explícitas de anulación.
4. **Escalabilidad:** paginación, índices basados en consultas, observabilidad y procedimiento de respaldos/restauración.

5. **Usabilidad:** accesibilidad, estados vacíos, formularios validados, confirmación para acciones destructivas y responsive testado.

Operación y continuidad

Responsabilidades operativas recomendadas

- **Administrador Supabase:** secretos, políticas, migraciones, funciones, usuarios privilegiados, límites/cuotas y copias.
- **Responsable clínico:** roles de la clínica, exactitud de catálogo y uso clínico; no comparte credenciales.
- **Responsable de aplicación:** releases de frontend, observabilidad, revisión de dependencias y recuperación de fallos.

Lista previa a producción

- ! Aplicar migrations 001–008 en entorno objetivo y revisar errores de ejecución.
- ! Probar políticas RLS con rol médico, secretaria, usuario desactivado y sesión sin perfil.
- ! Confirmar Realtime habilitado para las tablas requeridas.
- ! Validar alta/cierre de cuentas y recuperación/rotación de contraseña.
- ! Ejecutar build de producción y revisar rutas directas y actualización del navegador.
- ! Probar exportación a papel/PDF de informe médico, receta e informe contable.
- ! Definir política de respaldo, retención y prueba de restauración.
- ! Verificar que no se publique .env ni SERVICE_ROLE_KEY y que las variables VITE apunten al proyecto correcto.
- ! Documentar contacto y procedimiento ante incidente o indisponibilidad.

Supuestos de continuidad

La disponibilidad de Auth, base, funciones, canales realtime y almacenamiento del sistema depende del proyecto/proveedor Supabase y su región. La disponibilidad Netlify cubre el frontend estático, no garantiza la base de datos. El repositorio no demuestra una política de backup, SLA, RPO/RTO, monitorización o plan de desastre; deben determinarse fuera del código y comprobarse periódicamente.

15 · REFERENCIAS

Glosario y fuente de verdad

TÉRMINO	SIGNIFICADO EN ESTE DOCUMENTO
SPA	Aplicación de una página; el enrutamiento principal se resuelve en el navegador.
RLS	Row Level Security de PostgreSQL; políticas que determinan filas/operaciones permitidas bajo el rol de base de datos.
Edge Function	Función Deno alojada en Supabase para lógica backend, aquí usada para administración privilegiada.
JWT	Token de sesión firmado que identifica al usuario en llamadas a Supabase.
JSONB	Tipo PostgreSQL para documentos JSON consultables; usado para componentes flexibles de expediente/signos.
Anon key	Credencial pública de cliente Supabase; la autorización de datos debe aplicarse con Auth/RLS.
Service role	Credencial administrativa capaz de omitir RLS; secreto de backend.
HC-####	Identificador legible de historia clínica asignado por secuencia SQL.

Archivos consultados como evidencia

- `package.json`, `package-lock.json`
- `vite.config.js`, `netlify.toml`
- `.env.example`
- `src/App.jsx`, `src/main.jsx`
- `src/context/AuthContext.jsx`
- `src/lib/supabaseClient.js`, `dateUtils.js`
- `src/pages/`: login, Dashboard, pacientes, PacienteDetalle, Agenda, Pagos, Configuracion
- `src/components/`: sidebar, topbar, Modal, LogoMark
- `src/index.css`
- `supabase/migrations/001…008`
- `supabase/functions/create-user/index.ts`
- `supabase/functions/manage-user-access/index.ts`
- `supabase/functions/_shared/cors.ts`

> **Control de versión del documento:** edición 1.0 elaborada el 11/10/2026 a partir del estado del workspace SIGEC-26. Debe revisarse cuando cambien esquema SQL, permisos, funciones o flujos funcionales.

SIGEC Cloud · Clínica San Pedro

Documentación técnica interna · Edición 1.0 · 11 de octubre de 2026

El código y las migraciones del entorno desplegado son la autoridad final para la implementación.